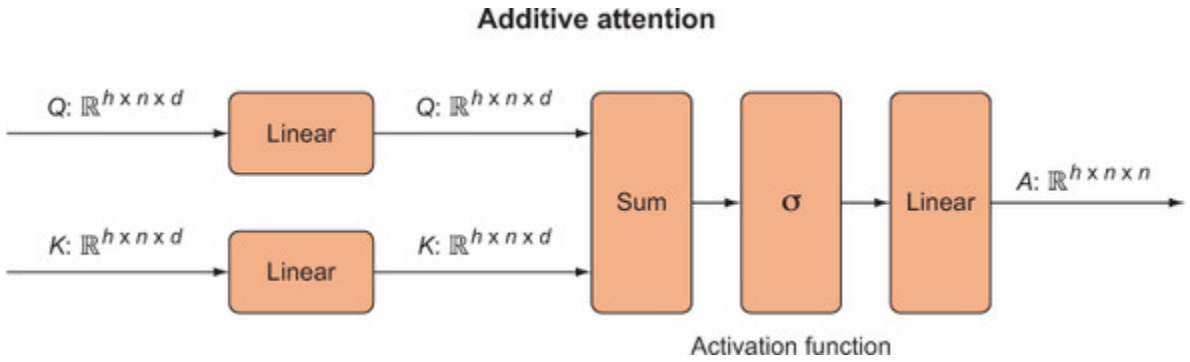
when we then pass it through the softmax function, which can easily saturate. By *saturate* we mean that when a particular value in an input vector is dramatically larger than other values in the vector, the softmax may assign all its probability mass to that single value, setting all the others to zero, or vice versa. This can make our gradients too large or too small for particular values and destabilize training.

Additive attention can solve this problem at the expense of introducing additional parameters. Instead of simply multiplying the Q and K tensors together, we'll instead pass them both through independent linear layers, add them together, and then apply an activation function, followed by another linear layer (figure 10.21). This allows for a more complex interaction between Q and K without as much risk of causing numerical instability, since addition will not exaggerate numerical differences like multiplication does. First we need to add three new linear layers for the additive attention.

Figure 10.21. Additive attention is an alternative to dot product attention that can be more stable. Instead of multiplying the queries and keys together, we first pass them independently through linear layers and then add them together, apply a nonlinear function, and pass through another linear layer to change the dimensionality.




```
>>> self.k_lin = nn.Linear(self.node_size, self.N)
>>> self.q_lin = nn.Linear(self.node_size, self.N)
>>> self.a_lin = nn.Linear(self.N, self.N)
```

In the forward method we define the actual computation steps for additive attention:

```
>>> A = torch.nn.functional.elu(self.q_lin(Q) + self.k_lin(K))
>>> A = self.a_lin(A)
```